
APC: Informações iniciais

Algoritmos de Programação de Computadores

Universidade de Brasília

Ciência da Computação

Isabela Honda

Monitora de APC

**Guia para quem nunca programou na vida e está entrando
no curso de APC.**

Com foco nos estudantes 60+.

Linguagem: **C99**

(C99 é uma versão da linguagem C)

*Utilizei IA para formatar o material em L^AT_EX e corrigir alguns *poucos* detalhes na escrita. Com essas
ressalvas, é um texto HUMANO!*

Tópicos

1	Materiais úteis	2
2	Onde praticar	2
3	Explicação básica	2
4	Como preparar o computador para programar	3
4.1	Opção 1: Editor de código + terminal (ideal)	3
4.2	Opção 2: Compilador online (mais fácil de usar)	4
4.3	Opção 3: Fazer diretamente na plataforma (não recomendado)	4
5	Terminal (para quem escolheu a Opção 1)	4
5.1	O que está ali?	4
5.2	Como criar um arquivo em C pelo terminal	4
5.3	Como abrir o seu arquivo no editor de código	5
5.4	Como escrever o código	5
5.5	Teste seu código	5
	Comando para compilar	5
	Comando para executar	5
6	Erros mais comuns	6
6.1	Erro de ortografia do código	6
6.2	Erro de lógica	6
6.3	Não escrever exatamente como a plataforma pediu	6
7	Ortografia básica da programação em C	6
7.1	Bibliotecas	6
7.2	Declarando as variáveis	7
7.3	Tipos derivados	9
7.4	Estruturas Condicionais	10
7.5	Estruturas de Repetição	10
7.6	Subalgoritmos (funções)	10
7.7	Matrizes	10
7.8	Manipulação de arquivos	10

Legenda de cores

- **Em rosa** estão os comandos de terminal.
- **Em verde** estão as linhas de código.

1 Materiais úteis

- Materiais de aula da professora Carla Castanho:
<https://carlacastanho.github.io/Material-de-APC/aulas.html>
- Videoaulas de C: [Linguagem C | Aula 01 - Esqueleto de um programa em C](#)

2 Onde praticar

- Beecrowd: <https://judge.beecrowd.com/>
- LeetCode: <https://leetcode.com/accounts/login/>
- Codeforces: <https://codeforces.com/>

3 Explicação básica

Antes de começar, estes são os termos que você vai ouvir com frequência:

Editor de código

Lugar do computador onde você escreve e edita seu código.

IDE Inclui as funcionalidades do editor de código, mas é um pouco mais completa. Também serve como compilador (ou interpretador, no caso de algumas outras linguagens), *debugger* e outros. O VSCode é uma IDE, ou seja, um editor de código com mais funcionalidades.

Compilar

Traduzir um código em linguagem humana para a linguagem de máquina. Por exemplo, fazer um código em C virar instruções binárias para que o computador consiga lê-lo.

Debugar

Corrigir erros do código.

Executar

Fazer um programa funcionar. Por exemplo, mandar escrever “Hello, world!” na tela do terminal.

Terminal

Lugar onde você escreve comandos para a máquina tentar compilar e executar. Por exemplo, mandar escrever algo na tela, criar um arquivo, mudar o nome de um arquivo. . .

Programadores mais experientes também usam o terminal para realizar instalações,

interações com o GitHub, programar no Nano, no Vim, e várias outras coisas. Mas esse não é o foco do curso agora.

Não tenha medo das mensagens de erro

É comum que o código não dê certo nos primeiros testes. Depois de mandar compilar, caso apareçam várias palavras e letras estranhas no terminal, provavelmente é o computador avisando que não conseguiu executar algo corretamente. Busque corrigir o erro no seu código para que essas mensagens não apareçam.

4 Como preparar o computador para programar

Os alunos de APC costumam seguir três caminhos principais. Escolha o que for mais confortável para você.

4.1 Opção 1: Editor de código + terminal (ideal)

Exemplos de editores de código:

- VSCode
- Bloco de Notas

Sobre o VSCode

O VSCode não é aceito por alguns professores, já que os alunos podem usar a IA (Copilot) que está inserida nele para colar código.

Você pode utilizar o terminal que já vem dentro do VSCode ou o terminal do computador para testar os programas. Peça ajuda para um monitor te mostrar onde ficam esses terminais.

Vai programar no seu computador pessoal?

É preciso instalar o aplicativo editor de código. Para o VSCode:

- Link de instalação: <https://code.visualstudio.com/download>
- Pesquisas úteis no YouTube:
 - [Clique aqui para acessar tutoriais de como instalar no Linux](#)
 - [Clique aqui para acessar tutoriais de como instalar no Windows](#)
- Com o app instalado, é necessário baixar a **extensão de C/C++**.

4.2 Opção 2: Compilador online (mais fácil de usar)

Exemplo: **Online GDB** — https://www.onlinegdb.com/online_c_compiler

1. No canto superior direito, selecione a linguagem “C”.
2. Escreva seu código.
3. Teste o código apertando em **Run**.

Não há necessidade de escrever os comandos de terminal!

Aperte em **Stop** em caso de travamento e tente novamente.

4.3 Opção 3: Fazer diretamente na plataforma (não recomendado)

Você pode escrever e testar seu código diretamente na plataforma de programação (por exemplo, no Beecrowd). Não é recomendado porque, dessa forma, não há como testar como o código funciona. Isso tira uma etapa muito grande do processo de aprendizado.

No entanto, é uma alternativa rápida e fácil para quando estiver com pressa de saber se acertou a questão!

5 Terminal (para quem escolheu a Opção 1)

5.1 O que está ali?

Costuma aparecer, no começo de cada linha, as informações de usuário e a pasta atual.

Exemplo no estilo Linux/Mac:

```
usuario@computador:~/Documentos/Projetos$
```

Exemplo no estilo Windows:

```
C:\Users\Usuario\Projetos>
```

Nesses exemplos, a pasta atual é “Projetos”.

5.2 Como criar um arquivo em C pelo terminal

Escolha um nome qualquer para o seu arquivo, **sem espaços e sem acentos**. Exemplo: aula1. Escreva assim no terminal e dê `Enter`:

```
touch aula1.c
```

- Um arquivo em C sempre termina em `.c`.
- `touch` é um comando para criar arquivos no terminal.
- Lembre-se de substituir “aula1” pelo nome escolhido.
- Seu arquivo será criado dentro da pasta indicada pelo terminal.

5.3 Como abrir o seu arquivo no editor de código

No editor de código de preferência, selecione a opção de abrir arquivo e clique no arquivo que você criou. Sempre é bom pedir ajuda para um monitor nesses momentos.

5.4 Como escrever o código

Estrutura básica (mais detalhes na Seção 7). Isso fica dentro do arquivo `.c`:

```
1 #include <stdio.h>
2
3 int main() {
4
5     return 0;
6 }
```

Depois de escrever seu programa, lembre-se de clicar em `Ctrl` + `S` para salvá-lo antes de partir para a compilação (caso sua IDE já não esteja salvando o código automaticamente).

5.5 Teste seu código

Comando para compilar

Escreva assim no terminal e dê `Enter`:

```
gcc -o aula1 aula1.c
```

Lembre-se de substituir “aula1” pelo nome escolhido.

Neste momento, não deve acontecer nada: o terminal só passa para a próxima linha. Caso apareçam várias coisas escritas, são mensagens de erro, o que significa que você deve voltar para o código e consertar alguma coisa (compilation error). Depois de consertar o código, salve o arquivo e escreva `gcc -o aula1 aula1.c` novamente. Dando tudo certo, você pode partir para o comando de executar.

Comando para executar

Escreva assim no terminal e dê `Enter`:

```
./aula1
```

Lembre-se de substituir “aula1” pelo nome escolhido.

Caso seu código tenha `scanf`, o terminal vai ficar esperando você digitar as entradas e dar `Enter`. Caso tenha `printf`, vai aparecer aquilo que você mandou escrever. Não necessariamente nessa ordem; depende de como o código foi escrito.

Só porque o programa executou, não significa necessariamente que está correto. É possível que ainda hajam erros de lógica a serem concertados, e a plataforma de programação como o Beecrowd aponta isso.

Resumo do ciclo de trabalho

Escrever o código → salvar (`Ctrl+S`) → compilar (`gcc -o aula1 aula1.c`) → executar (`./aula1`).

6 Erros mais comuns

6.1 Erro de ortografia do código

- Esquecer de usar “`;`” ao final das linhas.
- Usar o tipo de dado incorreto, como usar `%d` em vez de `%c` para identificar um `char`.

6.2 Erro de lógica

- Montar um *loop* (estrutura de repetição) incorretamente.
- Errar uma fórmula matemática.

6.3 Não escrever exatamente como a plataforma pediu

- Esquecer o `\n` na hora de fazer um `printf`.
- Escrever “Hello World” em vez de “Hello, World!”.

7 Ortografia básica da programação em C

7.1 Bibliotecas

Bibliotecas trazem funções¹ de C prontas e deixam a programação muito mais simples.

¹Funções são conjuntos de comandos que realizam uma tarefa específica. Exemplo: escrever um texto na tela (`printf`).

A biblioteca `stdio` (*standard input/output*) é a mais usada, e é declarada da seguinte maneira:

```
1 #include <stdio.h>
```

O “.h” remete a arquivo de cabeçalho (*header file*). Esse arquivo é sempre incluído (`include`) no **topo** do código.

A tabela a seguir apresenta as bibliotecas mais usadas no curso de APC e as funções que pertencem a elas.

#include <stdio.h>	#include <stdlib.h>	#include <math.h>	#include <string.h>
printf	malloc	sqrt(x)	strlen
scanf	calloc	pow(x, y)	
getchar	realloc	sin(x)	
fgets	free	cos(x)	
fopen	qsort	tan(x)	
fclose	rand		
fprintf	abs		
fscanf			
fread			
fwrite			

Atenção

No VSCode, cheque se a extensão de C/C++ está instalada para que o código funcione!

7.2 Declarando as variáveis

Declarar variáveis em C é reservar espaços de memória para as informações que serão usadas ao longo do programa, sejam elas números, caracteres, palavras...

Tipos de dados primitivos

Formato	Tipo	Descrição
<code>%d</code>	<code>int</code>	Números inteiros, sem casas decimais (32 bits).
<code>%f</code>	<code>float</code>	Números reais de precisão simples (32 bits), com casas decimais (geralmente umas 6 casas depois da vírgula).
<code>%lf</code>	<code>double</code>	Números reais de precisão dupla (64 bits). Suportam ainda mais casas decimais que o <code>float</code> (geralmente umas 15 casas depois da vírgula).
<code>%c</code>	<code>char</code>	Um único caractere (8 bits), como uma letra ou símbolo.

Você pode declarar uma variável e atribuir um valor para ela diretamente:

```
1 int numero = 10;
```

Ou você pode declarar e atribuir um valor depois:

```
1 int numero;  
2 // ( . . . )  
3 numero = 10;
```

Inteiro

```
1 int quantidade = 200;  
2 printf("%d\n", quantidade); // sai 200 no terminal  
3  
4 int volume;  
5 scanf("%d", &volume); // o terminal espera a escrita de um valor inteiro  
6 // o valor lido ficará armazenado no int volume
```

Float

```
1 float pi1 = 3.141592;  
2 printf("%f\n", pi1); // sai 3.141592 no terminal
```

Double

```
1 double pi2 = 3.141592653549793;
```

Char

```
1 char alguma_letra = 'a';
2 char algum_caractere = '@';
```

7.3 Tipos derivados

Tipo	Descrição
Ponteiros (<code>*</code>)	Reservam endereços de memória, o que permite que modifiquem dados indiretamente.
Vetores/Arrays (<code>[]</code>)	Conjuntos de dados do mesmo tipo.
Estruturas (<i>structs</i>)	Conjuntos de dados relacionados, que podem ser de tipos diferentes.

Exemplo: declarar um ponteiro e associá-lo a outro dado 'a'

```
1 int a = 10;
2 int *ponteiro;
3 ponteiro = &a; // associa o ponteiro ao endereço de 'a'
4
5 printf("Valor de a: %d\n", a);
6 printf("Endereço de a: %p\n", (void *)&a);
7 printf("Valor armazenado no ponteiro: %p\n", (void *)ponteiro);
8 printf("Valor apontado pelo ponteiro: %d\n", *ponteiro);
```

Exemplo: declarar um vetor

De inteiros:

```
1 int sequencia[30]; // suporta 30 números inteiros, cada um com um índice
```

De char:

```
1 char nome[50]; // suporta uma sequência de até 50 caracteres,
2               // o que pode formar palavras (strings)
```

Exemplo: struct

```
1 typedef struct {
2     int codigo;
```

```
3     char nome[30];
4     double peso;
5 } Nome_da_struct;
6
7 // ( . . . )
8
9 int main() {
10     Nome_da_struct nome;
11     // ( . . . )
12     return 0;
13 }
```

7.4 Estruturas Condicionais

A fazer.

7.5 Estruturas de Repetição

A fazer.

7.6 Subalgoritmos (funções)

A fazer.

7.7 Matrizes

A fazer.

7.8 Manipulação de arquivos

A fazer.